

Enterprise Skill Repository

Detailed Reference: Testing, Model Compatibility, Certification, and Security

This document expands the architecture into implementation-level guidance for building a trusted skill platform.

Executive Summary

- A Skill Repository should be a trusted capability registry, not only a package store.
- Every skill version should include strict contracts, compatibility declarations, security metadata, and certification evidence.
- A production-grade system separates discovery, authorization, validation, and runtime enforcement.
- Core principle: discovery != authorization; compatibility != certification; certification != unrestricted execution.

1) What to Store in a Skill Version

- Identity: skill id, semantic version, owner, changelog pointer, and artifact digest.
- Contract: input schema, output schema, behavior constraints, and error taxonomy.
- Implementation: prompts, code, connector interface definitions, adapter rules.
- Examples: representative valid/invalid request samples and expected responses.
- Tests: unit/integration/regression suites with fixtures and expected outcomes.
- Compatibility declarations: runtime/dependency matrix and model capability requirements.
- Security declarations: risk level, permissions, secret references, approval policy id, network scope.

2) Testing Architecture (First-Class)

- Repository stores test contracts; validation pipeline executes tests and publishes evidence.
- Level 1 static checks: manifest/schema/security policy/dependency declarations/signature checks.
- Level 2 unit tests: deterministic, isolated logic checks without external dependencies.
- Level 3 integration tests: real dependency behavior (auth, retries, syntax, network failures).
- Level 3 compatibility matrix: dependency versions (for example DB 10.4/10.6/10.11).
- Level 4 regression: permanent tests for previously fixed bugs and incident replay cases.
- Validation should output immutable run artifacts and attach evidence links to the registry.

3) Model Compatibility (Required)

- Yes, model testing should be considered a first-class dimension.
- Do not fork skill IDs by model by default; declare capability requirements and measure empirical model outcomes.
- Required capability fields: tool calling, structured output, minimum context window, minimum reasoning level, modalities.

- Evaluate tool selection accuracy, argument completeness, schema conformance, hallucination resistance, and task success.
- Use canary validation for model upgrades and keep a failure replay suite from production incidents.
- Do not test every skill against every model; use capability profile matching before expensive evaluations.

4) Certification Lifecycle

- Use a lifecycle such as DRAFT -> VALIDATING -> TESTED -> CERTIFIED -> PRODUCTION -> DEPRECATED.
- Certification should be scoped to environment and model profiles with explicit timestamps and evidence.
- No automatic inheritance across versions; each version must pass full required gates.
- Execution policy can enforce: only CERTIFIED versions for the current environment and model profile.

5) Security Design (Defense in Depth)

- Apply defense in depth: identity, authorization, approvals, secure runtime, secrets isolation, and auditing.
- Use RBAC as baseline and ABAC for contextual controls (risk, environment, amount, tenant, time).
- Require re-authorization at execution time and approvals for high-risk skills.
- Never embed credentials in skills; issue short-lived runtime credentials with independent secret authorization.
- Sandbox execution with network allowlists, filesystem isolation, and least-privilege connector permissions.
- LLM can propose actions, but policy engine must authorize actions.
- Capture immutable audit trails without leaking secrets or sensitive raw payloads.
- Enforce package supply-chain security: secret scan, malware scan, signing, and integrity verification.

6) Recommended Selection Flow

- Capability match -> Environment compatibility -> Model compatibility -> Certification -> Security policy -> Reliability ranking -> Execute.

7) Operational Separation of Concerns

- Skill Repository: what exists.
- Validation/CI: what is proven to work.
- Policy and Approval Services: who may execute and under what controls.
- Skill Runtime: how execution is constrained.
- Audit and Observability: what happened and how well it performed.

8) Detailed Security Controls

Identity and Context

- Carry tenant_id, user_id, agent_id, session/request id, environment id in every authorization and audit event.
- Preserve both user and agent identity so actions are attributable end-to-end.

Authorization

- RBAC grants coarse rights (for example finance-agent can use payroll.read).
- ABAC enforces context gates (amount thresholds, environment restrictions, time windows).
- Evaluate authorization twice: at selection time and immediately before execution.

Risk and Approval

- Classify skill actions by risk level (L0 informational -> L4 critical).
- Require human approvals for sensitive and critical actions.
- Define approval TTL, approver roles, and minimum approver count.

Secrets and Credentials

- Skill packages should store only secret references, never secret values.
- Runtime requests short-lived credentials from a secrets manager.
- Secret access has its own policy checks independent of skill execution rights.

Execution Isolation

- Run skill logic in sandboxed containers or equivalent isolated runtimes.
- Apply network egress allowlists per skill.
- Enforce filesystem isolation and resource quotas (CPU, memory, timeout).

Prompt Injection Boundary

- Treat retrieved external text as untrusted input.
- Never allow model output to bypass policy and approval boundaries.

Audit and Forensics

- Store immutable events: principal, skill id/version, decision, resource, result, duration.
- Redact or hash sensitive fields; do not persist credentials or raw personal data.

9) Example Policy Envelope

```
{ principal: { user: "user-123", agent: "finance-agent" }, action: "execute", skill: {
  id: "payroll.run", version: "3.2.0" }, resource: { system: "Payroll", environment:
  "prod" }, context: { tenant: "company-a", amount: 25000 } }
```

Policy decision result should be one of: ALLOW, DENY, or REQUIRES_APPROVAL.

10) Example Validation Record

```
{ skill_id: "jira.create_ticket", version: "2.1.0", status: "CERTIFIED", tests: { unit:
  { passed: 124, failed: 0 }, integration: { jira_v9: "PASS", jira_v10: "PASS" },
  regression: { passed: 37, failed: 0 } }, model_eval: { model_a: { task_success: 0.98 },
  model_b: { task_success: 0.95 } }, last_validated_at: "2026-08-08T10:15:00Z" }
```

11) Operational Checklists

Release Readiness Checklist

- Manifest and schemas pass validation.
- Unit tests pass at required threshold.
- Integration matrix passes for declared dependency versions.
- Regression suite passes including historical incidents.
- Model evaluation passes minimum quality bars.
- Security scans, signing, and integrity checks pass.
- Observability and audit event fields are complete.

Production Admission Checklist

- Certification status is CERTIFIED for current env/model profile.
- Policy and approval bindings are attached and active.
- Secret references resolve to valid short-lived credentials.
- Runtime sandbox and network rules are enforced.
- Rollback path and deprecation policy are defined.

Monitoring KPIs

- Task success rate by model and skill version.
- Tool-call correctness rate.
- Schema conformance rate.
- p95 end-to-end latency.
- Authorization deny ratio and approval latency.
- Regression failure count and drift incidents.

Final Recommendation

Treat trust as a computed, continuously validated state. Build a pipeline where capability fit, environment fit, model fit, certification evidence, and security authorization are all mandatory gates before execution.